📝 Mock DS — Practical part (on PC) — Python for Data Science

Duration: 2 h · **Total:** /20 · Documents not allowed · One notebook to submit: `NAME_Surname_DS.ipynb`

Rules — Write your code in the empty cells. Each result must be **visible** (print / display). Each cleaning decision needs a **one-line comment** explaining *why*. Answer written questions in a Markdown cell or in a comment.

Exercise	Topic	Points
1	Python basics — pharmacy stock	4
2	NumPy — daily visits	4
3	pandas — PharmaPlus sales: load, explore, clean, analyse, export	12

Files: `data/pharmacy_sales.csv` (sales of the **PharmaPlus** pharmacy chain, 2025) and `data/medicines.csv` (catalogue). In Colab, upload both files (they go to `/content`).

Exercise 1 — Python basics: pharmacy stock (4 pts)

Use only plain Python (no pandas, no NumPy).

In [ ]:
```
stock = {"Doliprane": 40, "Vitamin C": 0, "Sunscreen": 12, "Cough syrup": 4, "Thermometer": 3, "Omega 3": 7}
prices = {"Doliprane": 3.9, "Vitamin C": 6.9, "Sunscreen": 29.0, "Cough syrup": 7.2, "Thermometer": 25.0, "Omega 3": 17.5}
```

1.1 (1 pt) With a `for` loop on `stock.items()`, print one line per product: `Doliprane -> 40 units`. Add `(OUT OF STOCK)` when the quantity is 0 and `(LOW)` when it is between 1 and 4.

In [ ]:
```
# 1.1
```

1.2 (1 pt) Write a function `stock_value(stock, prices)` that **returns** the total value of the stock (Σ quantity × price), rounded to 2 decimals. Print the result with an f-string: `Stock value: ... TND`.

In [ ]:
```
# 1.2
```

1.3 (1.5 pt) Write a function `sell(stock, name, qty)` that:

- returns `False` and prints `Unknown product` if `name` is not in the stock;

- returns `False` and prints `Not enough stock` if there are fewer than `qty` units;
- otherwise removes `qty` units from the stock and returns `True`.

Test it with: `sell(stock, "Sunscreen", 5)`, `sell(stock, "Thermometer", 10)`, `sell(stock, "Aspirin", 1)`, then print `stock["Sunscreen"]`.

In [ ]:
```
# 1.3
```

1.4 (0.5 pt) Build the **sorted list** of products to reorder (quantity < 5) and print it with its length.

In [ ]:
```
# 1.4
```

Exercise 2 — NumPy: daily visits (4 pts)

The pharmacy of Sousse counted its customers every day for 4 weeks (Monday → Sunday).

In [ ]:
```
import numpy as np

days = np.array(["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"])
visits = np.array([95, 110, 102, 120, 135, 160, 70,
                   98, 115, 99, 125, 140, 172, 65,
                   101, 108, 97, 118, 150, 168, 80,
                   90, 112, 105, 130, 145, 181, 75])
```

2.1 (1 pt) Print the `shape`, `size`, `ndim` and `dtype` of `visits`. Reshape it into a 2D array `weeks` with one row per week. Print the shape of `weeks`. In a comment: what would `visits.reshape((5, 6))` do, and why?

In [ ]:
```
# 2.1
```

2.2 (1 pt) Without any loop, compute: the total number of visits, the average visits **per week** (one value per week) and the average visits **per weekday** (one value per day of the week, rounded to 1 decimal).

In [ ]:
```
# 2.2
```

2.3 (1 pt) Find the day with the most visits over the 4 weeks: print its value, the **week number (1–4)** and the **day name**. *(Hint: `argmax()` on the 1D array, then `//` and `%`.)*

In [ ]:
```
# 2.3
```

2.4 (1 pt)

1. How many days had **more than 130** visits?

2. Display the names of the weekdays whose average (from 2.2) is **≥ the overall average**.
3. In a comment, give the `dtype` of `np.array([1, 2.5, True])` and of `np.array([1, "2", 3.0])`, and explain the rule.

In [ ]:
```
# 2.4
```

Exercise 3 — pandas: PharmaPlus sales (12 pts)

Scenario. PharmaPlus exports the sales of its 8 pharmacies every night. The finance team needs a **trusted** dataset and a few KPIs. The file is dirty. Never modify the raw DataFrame: work on a copy.

`pharmacy_sales.csv` columns: `sale_id, sale_date, pharmacy_city, med_id, quantity, unit_price, payment, customer_age, prescription` `medicines.csv` columns: `med_id, med_name, category, cost_price, list_price`

Business rules given by the manager

- One sale line contains between **1 and 50** units. Anything else is a data-entry error.
- A customer is between **18 and 100** years old. Other ages are errors → treat them as missing.
- Every sold `med_id` must exist in the catalogue.

3.1 Load and explore (2 pts)

1. Import pandas, load the two files into `sales_raw` and `meds`.
2. Display the first 5 rows, the shape, `info()`, `describe()` and the number of unique values per column of `sales_raw`.
3. Display the distinct values of `pharmacy_city` and the frequencies of `quantity`.
4. In a Markdown cell, list **at least 4** data-quality problems you see.

In [ ]:
```
# 3.1
```

In [ ]:
```

```

Your answer:

3.2 Clean (5 pts) — create `sales` = a cleaned copy of `sales_raw`

Do the steps **in this order**, print the number of rows after each step that removes rows, and comment every decision.

1. Compute the **percentage of missing values per column** (sorted descending).
2. Remove the exact duplicates.
3. Clean `pharmacy_city` so that exactly **8 cities** remain.
4. Convert `sale_date` to datetime (invalid dates → missing) and drop the sales without a valid date. *Why drop instead of fill?*
5. Apply the quantity rule, then convert `quantity` to `int64`.
6. Remove the sales whose `med_id` is not in the catalogue.
7. Ages outside [18, 100] become missing; then fill all missing ages with the **median** age. *Why the median and not the mean?*
8. Fill missing `payment` with `"Unknown"`.
9. Fill missing `unit_price` with the **median price of the same medicine** (`groupby` + `transform`).
10. Validate with `assert` that no missing value is left, and print a cleaning report: raw rows, clean rows, % removed.

In [ ]:
```
# 3.2 — 1. missing values
```

In [ ]:
```
# 2. duplicates
```

In [ ]:
```
# 3. city
```

In [ ]:
```
# 4. dates
```

In [ ]:
```
# 5. quantity rule
```

In [ ]:
```
# 6. unknown medicines
```

In [ ]:
```
# 7. ages
```

In [ ]:
```
# 8-9. payment and price
```

In [ ]:
```
# 10. validation and report
```

Your answer:

3.3 Analyse (4 pts)

1. Merge `sales` with `meds` (keep all the sales) into `df`. Create `revenue = quantity × unit_price` and `profit = revenue − quantity × cost_price`. Print total revenue, total profit and the global margin (%).
2. Revenue by city, sorted descending, with each city's **share of total revenue (%)**. Which city is first?
3. Top 5 medicines by revenue (name + revenue).
4. For each `category`: number of sales, revenue, profit and margin %. Which category is the most profitable? Which has the best margin?
5. Which **month** has the highest revenue? Compare the revenue of the *Cold & Flu* category in winter months (12, 1, 2) and summer months (6, 7, 8).

6. Average basket (`revenue`) per `payment` method, and the **% of sales with a prescription**.
7. Sales from **Tunis**, category **Antibiotics**, with `quantity >= 3` : count them with a boolean mask **and** with `query()` — same result?
8. Create `age_group` with `pd.cut` : `18-29` , `30-49` , `50-64` , `65+` and give the revenue per group.

In [ ]:
```
# 3.3 – 1
```

In [ ]:
```
# 2
```

In [ ]:
```
# 3
```

In [ ]:
```
# 4
```

In [ ]:
```
# 5
```

In [ ]:
```
# 6
```

In [ ]:
```
# 7
```

In [ ]:
```
# 8
```

3.4 Export and conclusion (1 pt)

1. Export `df` to `pharmacy_clean.csv` without the index, reload it with `parse_dates=["sale_date"]` and check that the shape is identical.
2. Write a **3-line conclusion** for the manager (data quality + 2 key insights).

In [ ]:
```
# 3.4
```

Your answer: